H5: Loops deel 1

Labo 1 · week 5 · switch, while en do while

SEMESTER 1 · BASIS PROGRAMMEREN

Vandaag

1. `switch`: het laatste stukje van H4, voor als je veel `else if`'s hebt
2. **Waarom loops?** Code herhalen zonder te kopiëren
3. `while`: herhalen zolang een voorwaarde waar is
4. `do while`: eerst doen, dan controleren
5. **Alles samen**: een menu dat blijft terugkomen

😈 *Vandaag: loops. Vandaag: loops. Vandaag: loops. Sorry, iemand vergat de teller te verhogen.*

switch

Het probleem

Een menu met een lange reeks `else if`'s:

```
1  Console.WriteLine("Kies: 1) afbreken  2) opslaan  3) laden");
2  int option = Convert.ToInt32(Console.ReadLine());
3
4  if (option == 1)
5  {
6      Console.WriteLine("Afbreken gekozen");
7  }
8  else if (option == 2)
9  {
10     Console.WriteLine("Opslaan gekozen");
11 }
12 else if (option == 3)
13 {
14     Console.WriteLine("Laden gekozen");
15 }
16 else
17 {
18     Console.WriteLine("Onbekende keuze");
19 }
```

Drie keer `else if`: het programmeursequivalent van "euh... en ook nog...".

Hetzelfde met `switch`

```
 1  switch (option)
 2  {
 3      case 1:
 4          Console.WriteLine("Afbreken gekozen");
 5          break;
 6      case 2:
 7          Console.WriteLine("Opslaan gekozen");
 8          break;
 9      case 3:
10          Console.WriteLine("Laden gekozen");
11          break;
12      default:
13          Console.WriteLine("Onbekende keuze");
14          break;
15  }
```

`switch`: *geen spelconsole, wel een stuk overzichtelijker.*

De regels

- Elke `case` eindigt met `break`. Vergeet je die, dan krijg je fout **CS0163**.
- Een `case`-waarde is een **constante** (een literal zoals `1` of `"ja"`), geen variabele. Anders: fout **CS9135**.
- De `case`-waarden hebben **hetzelfde type** als de waarde die je test.
- `default` is optioneel, maar meestal een goed idee.



Vergeet je `break`, dan weigert C# te compileren. Eindelijk iemand die je verplicht pauze te nemen.

Meerdere waarden, dezelfde code

Zet lege `case`'s **onder elkaar**. Dat heet **fallthrough**:

```
 1  switch (option)
 2  {
 3      case 1:
 4          Console.WriteLine("Afbreken gekozen");
 5          break;
 6      case 2:
 7      case 3:
 8          Console.WriteLine("Laden of opslaan gekozen");
 9          break;
10      default:
11          Console.WriteLine("Onbekende keuze");
12          break;
13  }
```

Case 2 en case 3 delen hun code. Carpoolen, maar dan voor programmeurs.

Ook met strings

```
1  switch (day)
2  {
3      case "zaterdag":
4      case "zondag":
5          Console.WriteLine("Weekend!");
6          break;
7      case "woensdag":
8          Console.WriteLine("Halverwege!");
9          break;
10     default:
11         Console.WriteLine("Gewoon werken.");
12         break;
13 }
```

`day`	Output
`"zaterdag"`	`Weekend!`
`"woensdag"`	`Halverwege!`
`"maandag"`	`Gewoon werken.`



Weekend detecteren, deel 2. Nu met `switch`.

Waarom loops?

Terug naar boven

Met `if` kan je code **overslaan**. Wat we nog niet konden: **terug naar boven** gaan om iets opnieuw te doen.

> "Voer volgende code uit tot de gebruiker een bepaalde waarde invoert."

> **TIP**
>
> Zie je dezelfde regels code onder elkaar staan, bijvoorbeeld door copy-paste? Dan kan het bijna zeker korter met een loop.

Tien keer dezelfde regel kopiëren is geen programmeren. Dat is strafwerk.

Soorten loops

- **Definite** of **counted** loop: je weet vooraf hoe vaak. *"Toon de getallen van 1 tot 100."*
- **Indefinite** of **sentinel** loop: je weet het vooraf niet. *"Voer getallen in, typ 0 om te stoppen."*
- **Oneindige** loop: stopt nooit. Soms gewenst, vaak een bug.

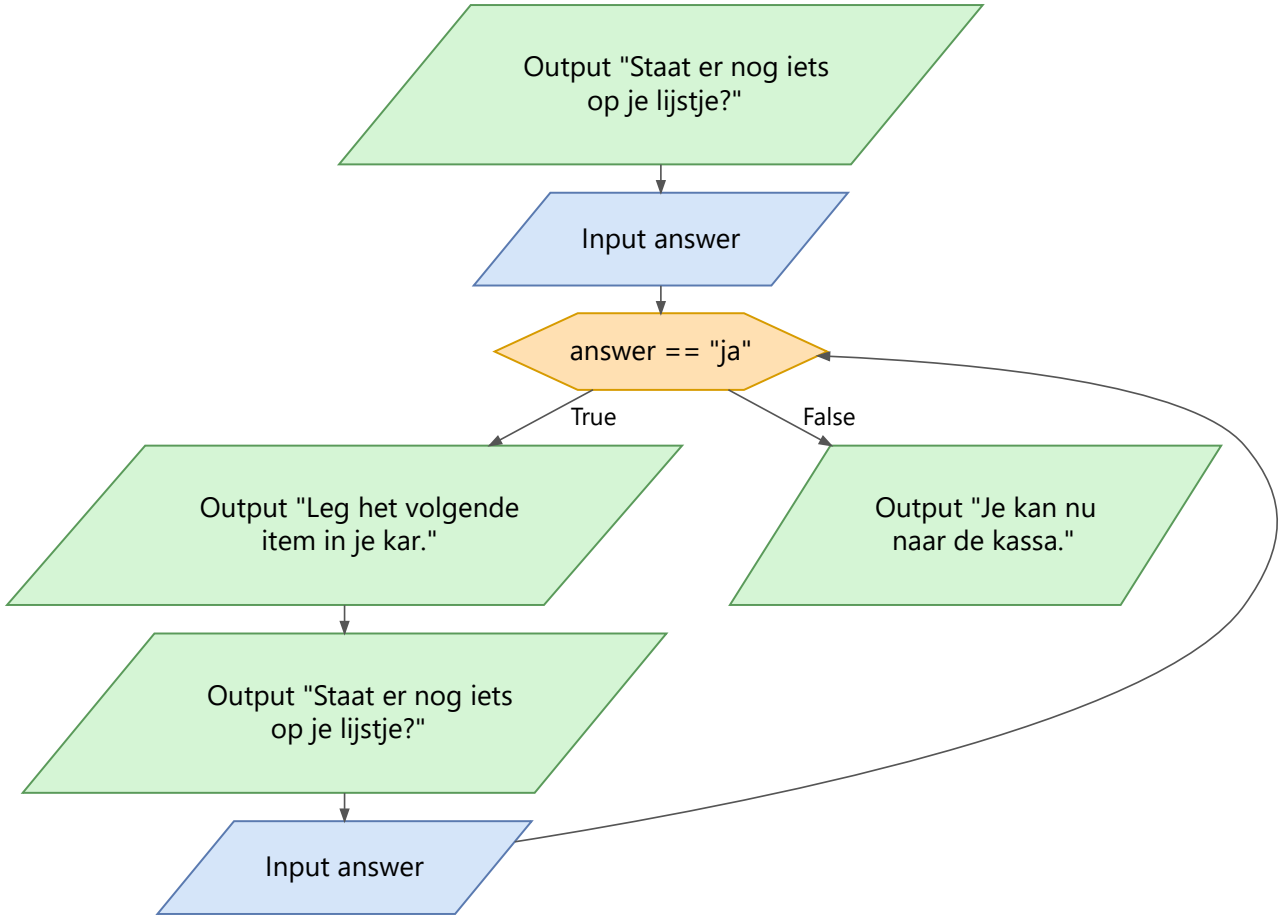
In C#: `while` (0 of meer keer), `do while` (minstens 1 keer) en `for` (volgende week).



Een oneindige loop is zoals de discussie over ananas op pizza.

while

Boodschappen doen



Zolang er iets op je lijstje staat... En er staat altijd chocolade op.

In C#

```
1 Console.WriteLine("Staat er nog iets op je lijstje? (ja/nee)");
2 string answer = Console.ReadLine();
3 while (answer == "ja")
4 {
5     Console.WriteLine("Leg het volgende item in je kar.");
6     Console.WriteLine("Staat er nog iets op je lijstje? (ja/nee)");
7     answer = Console.ReadLine();
8 }
9 Console.WriteLine("Je kan nu naar de kassa.");
```

- De voorwaarde wordt **vóór** elke herhaling gecontroleerd.
- Is ze meteen `false`, dan wordt de code **nul keer** uitgevoerd.
- **Binnen** de loop moet iets veranderen, anders stopt hij nooit.

Tellen met `while`

Wat verschijnt er op het scherm? 🤔

```
1  int count = 0;
2  while (count < 5)
3  {
4      count++;
5      Console.WriteLine(count);
6  }
```

```
1
2
3
4
5
```

Waarom ook **5**? Bij `count == 4` is `4 < 5` nog `true`. Binnen de loop wordt het dan eerst 5, en pas daarna getoond.

Tot 5 tellen: het enige wat elke programmeur foutloos kan. Na wat oefening.

Tot de gebruiker stopt

Een **sentinel loop**: herhalen tot een speciale waarde ingevoerd wordt.

```
1  int words = 0;
2  Console.WriteLine("Typ een woord (stop om te stoppen):");
3  string input = Console.ReadLine();
4  while (input != "stop")
5  {
6      words++;
7      input = Console.ReadLine();
8  }
9  Console.WriteLine($"Je typte {words} woorden.");
```

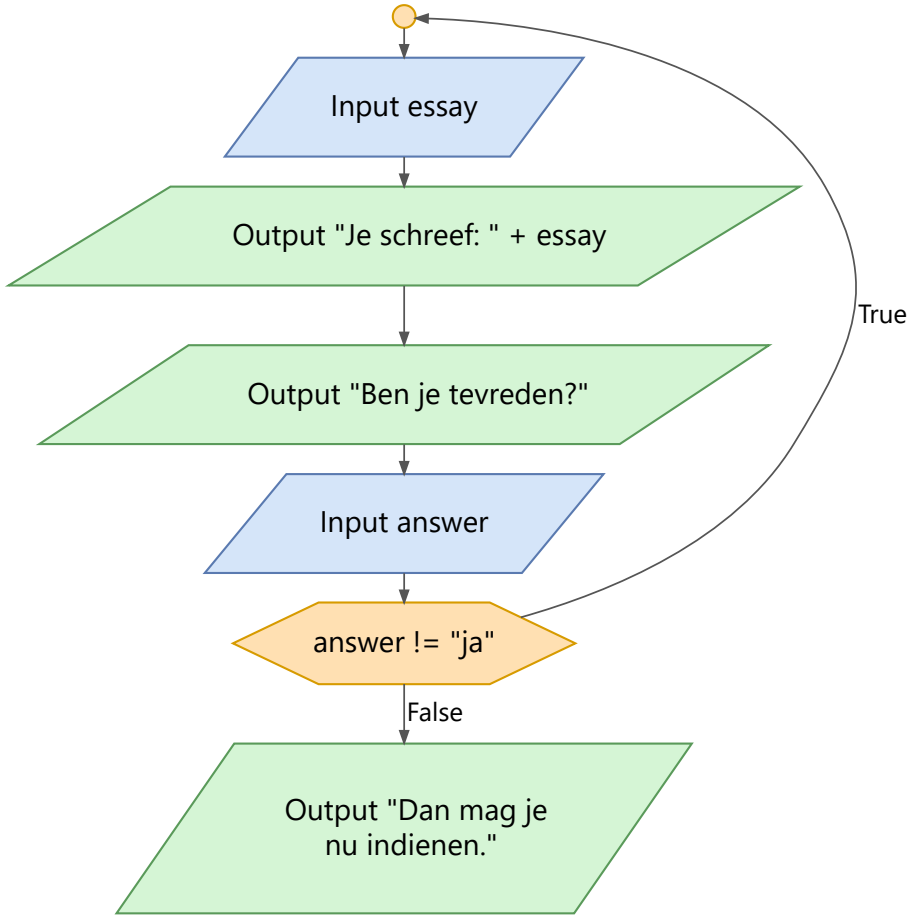
Patroon: **één keer lezen vóór** de loop, en **opnieuw lezen** als laatste stap **in** de loop.



"stop" typen om te stoppen. Werkt bij programma's. Minder bij een kleuter die "waarom?" vraagt.

do while

Een opstel schrijven



Eerst schrijven, dan nalezen. De volgorde die elke leerkracht graag ziet. En zelden krijgt.

In C#

```
 1  string answer;
 2  do
 3  {
 4      Console.WriteLine("Typ je opstel:");
 5      string essay = Console.ReadLine();
 6      Console.WriteLine($"Je schreef: {essay}");
 7      Console.WriteLine("Ben je tevreden? (ja/nee)");
 8      answer = Console.ReadLine();
 9  } while (answer != "ja");
10  Console.WriteLine("Dan mag je nu indienen.");
```

⚠ **WAARSCHUWING**

Achter de `while (...)` van een `do while` staat een **puntkomma**. Vergeet je die, dan krijg je fout CS1002. Bij een gewone `while` staat er **geen**.

Die puntkomma vergeet iedereen één keer. Daarna nooit meer. Of twee keer.

`while` of `do while`?

```
 1  int number = 10;
 2  while (number < 5)
 3  {
 4      Console.WriteLine($"while: {number}");
 5      number++;
 6  }
 7
 8  number = 10;
 9  do
10  {
11      Console.WriteLine($"do while: {number}");
12      number++;
13  } while (number < 5);
```

```
do while: 10
```

`while`: **0 of meer** keer. `do while`: **minstens 1** keer.

`while` kijkt voor hij springt. `do while` springt, en kijkt dan.

Alles samen

Een menu dat terugkomt

```
string choice;
do
{
    Console.WriteLine("1) Opslaan  2) Laden  3) Stoppen");
    choice = Console.ReadLine();
    switch (choice)
    {
        case "1":
            Console.WriteLine("Opgeslagen.");
            break;
        case "2":
            Console.WriteLine("Geladen.");
            break;
        case "3":
            Console.WriteLine("Tot ziens!");
            break;
        default:
            Console.WriteLine("Onbekende keuze");
            break;
    }
} while (choice != "3");
```

Het menu in actie

```
1) Opslaan  2) Laden  3) Stoppen
1
Opgeslagen.
1) Opslaan  2) Laden  3) Stoppen
3
Tot ziens!
```

- `do while`: het menu verschijnt **minstens één keer**
- `switch`: elke keuze krijgt haar eigen `case`
- de loop stopt pas als `choice` gelijk is aan `"3"`

Een menu dat blijft terugkomen tot je “stoppen” kiest. Zoals reclame op YouTube.

Samengevat

Wat	Hoe	Hoe vaak?
één waarde, veel vaste gevallen	`switch` + `case` + `break`	
herhalen zolang iets waar is	`while (...) { }`	0 of meer
eerst doen, dan controleren	`do { } while (...);`	minstens 1

> **BELANGRIJK**
>
> In een loop moet **iets veranderen** dat de voorwaarde beïnvloedt. Anders stopt hij nooit.

Je kan nu dingen herhalen. Je kan nu dingen herhalen. Oké, we stoppen.

Aan de slag 💻

1. **Keuzemenu's maken** (H4): een hoofdmenu en een submenu met `switch`
2. **CountDown**: aftellen van een getal tot 1, en dan "Start!"
3. **Password**: vraag een wachtwoord tot het juist is, en tel de pogingen
4. **Average**: gemiddelde van ingevoerde getallen, stoppen bij 0
5. **Lions**: hoe lang duurt het tot 50 leeuwen 100 leeuwen worden?
6. **NumberOfDigits**: tel de cijfers van een getal, zonder het als tekst te behandelen

 `while (!done) { Practice(); }`